

Second Order Neural Network Optimization: Meta Analysis

Jeshwanth Challagundla*
University of Texas at Arlington
Texas, USA
jeshwanth.challagundla@mavs.uta.edu

Mantek Singh
Liverpool John Moores University
Liverpool, England
mantek.singh2@gmail.com

Vivek Tiwari
Amazon
CA, USA
vivektiware1986@gmail.com

Siddharth Raina
Carnegie Mellon University
PA, USA
srainal@alumni.cmu.edu

Abstract—Second-order optimization algorithms have garnered significant interest in deep learning due to their ability to leverage curvature information, leading to faster convergence and enhanced generalization compared to first-order methods. This paper provides a comprehensive review of recent advancements in second-order optimization techniques tailored for deep neural networks, emphasizing their scalability, efficiency, and applicability across various tasks. We categorize these methods into Quasi-Newton methods, Generalized Gauss-Newton methods, and Parameter Subset Optimization methods, each offering unique strategies to balance computational cost and the advantages of second-order information. Despite their potential, these techniques face challenges such as high computational and memory demands, sensitivity to hyperparameters, and integration complexities in large-scale models. We discuss the innovative approaches employed by these algorithms to address these challenges, including structured approximations and adaptive updates, and provide insights into their theoretical foundations and empirical evaluations. The analysis highlights the critical role of second-order methods in advancing neural network optimization and outlines future research directions aimed at enhancing their scalability and practical adoption in deep learning applications.

Keywords—Second-order optimization, deep learning, neural networks, Generalized Gauss-Newton, Quasi-Newton, Newton's method.

I. INTRODUCTION

Deep learning has transformed fields such as computer vision and natural language processing by enabling neural networks to learn complex patterns from large datasets. The training of these models relies on optimization algorithms that minimize a loss function. First-order methods, like Stochastic Gradient Descent (SGD) [5], [48] and its adaptive variants, are commonly used due to their simplicity and scalability. However, these methods often suffer from slow convergence, sensitivity to hyperparameters, and inefficiencies in navigating complex, high-dimensional loss landscapes due to their reliance solely on gradient information.

Second-order optimization techniques offer a promising alternative by incorporating curvature information, using Hessians or approximations to adjust updates more effectively. These methods leverage both gradient and curvature data to

accelerate convergence, avoid poor local minima, and reduce hyperparameter sensitivity. Despite their advantages, traditional second-order methods, such as Newton's method, are often impractical in deep learning due to the computational and memory costs of calculating full Hessians.

To address these challenges, various second-order algorithms have been developed that approximate curvature information in more efficient ways. These algorithms can be grouped into three main categories: Quasi-Newton Methods, which approximate Hessians using gradient information; Generalized Gauss-Newton Methods, which efficiently capture second-order data through structured approximations like Kronecker-factored matrices; and Parameter Subset Optimization Methods, which selectively apply second-order updates to specific subsets of parameters.

This paper reviews these second-order optimization techniques, examining their theoretical foundations, practical implementations, and unique advantages in neural network training. By addressing the inherent challenges of second-order optimization, these methods provide a pathway to more efficient, scalable, and adaptive deep learning models.

II. SECOND-ORDER OPTIMIZATION ALGORITHMS

Neural network training algorithms can be broadly categorized into first-order and second-order methods [2], primarily distinguished by the type of information they utilize for parameter updates. First-order methods, such as Stochastic Gradient Descent (SGD) and its variants [46], [47], rely solely on the gradient of the loss function, which provides the direction of the steepest descent. These methods are computationally efficient per iteration and scale well with large datasets [12], making them the standard for training deep neural networks. However, they often struggle with slow convergence, especially in regions of the loss landscape with poor curvature, such as flat valleys or saddle points.

Second-order methods, such as Newton's method and Quasi-Newton methods, enhance optimization by utilizing both the gradient and the Hessian matrix, which encodes curvature

information about the loss surface. This additional information allows second-order methods to adjust step sizes more effectively, leading to potentially faster convergence and more precise parameter updates. However, these advantages come at the cost of higher computational requirements, particularly the need to compute, store, and invert the Hessian or its approximations, making direct application of these methods challenging in large-scale deep learning scenarios. Despite these challenges, second-order methods form the foundation for advanced optimization techniques that strive to balance computational efficiency with improved convergence.

A. Newton's Method

Newton's method is a foundational second-order optimization technique that uses curvature information to adjust the search direction, making it more effective than first-order methods in many scenarios. The method directly employs the Hessian matrix to make informed updates, based on a second-order Taylor series approximation of the objective function around the current point $\mathbf{x}_k$:

$$f(\mathbf{x}) \approx f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^T (\mathbf{x} - \mathbf{x}_k) + \frac{1}{2} (\mathbf{x} - \mathbf{x}_k)^T \nabla^2 f(\mathbf{x}_k) (\mathbf{x} - \mathbf{x}_k), \quad (1)$$

where $\nabla f(\mathbf{x}_k)$ is the gradient vector, and $\nabla^2 f(\mathbf{x}_k)$ is the Hessian matrix at $\mathbf{x}_k$. The update rule derived from this approximation is:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{H}_k^{-1} \nabla f(\mathbf{x}_k), \quad (2)$$

where $\mathbf{H}_k$ is the Hessian matrix. This update uses both gradient and curvature information to adjust the estimate, allowing the method to take larger steps in directions of low curvature and smaller steps in directions of high curvature.

Newton's method is integral to second-order optimization techniques due to its precise adjustments based on curvature, but its practical application in deep learning is limited by significant computational challenges.

B. Challenges of Newton's Method in Deep Neural Networks

Despite its advantages, Newton's method faces several major challenges when applied to deep neural networks:

- **Hessian Computation and Storage:** Computing the full Hessian matrix requires $O(n^2)$ storage and $O(n^3)$ computation time, which is infeasible for networks with millions of parameters.
- **Hessian Inversion:** Even when the Hessian is computed, inverting it is computationally expensive and often numerically unstable, especially in poorly conditioned matrices.
- **Non-convexity Issues:** The non-convex nature of neural network loss landscapes can lead to Hessians with negative eigenvalues, causing Newton's method to make updates that increase the loss.
- **Sensitivity to Initialization:** Newton's method is highly sensitive to the starting point, and poor initialization can lead to divergence, particularly in complex loss landscapes.

- **Scalability:** The method's requirement to recompute the Hessian and solve a system of linear equations at each step makes it impractical for large-scale or real-time applications.

These challenges necessitate the development of more practical second-order methods that approximate or selectively use curvature information, making them suitable for modern deep learning.

C. Categories of Second-Order Methods

To address the limitations of Newton's method, various second-order approaches have been developed that use approximations or selective applications of curvature information. These methods can be grouped into three main categories:

- **Quasi-Newton Methods:** These methods approximate the Hessian matrix using gradient information, avoiding the direct computation of second derivatives. Prominent algorithms include BFGS, L-BFGS, and stochastic adaptations.
- **Generalized Gauss-Newton Methods:** These methods capture curvature along specific residuals of the loss function, using structured approximations to reduce computational overhead. Key algorithms include Shampoo, K-FAC, EK-FAC, and AdaHessian.
- **Parameter Subset Optimization Methods:** These methods focus second-order updates on key subsets of parameters, such as learning rates or specific weights, balancing computational cost and performance. Notable examples are Multiple Optimal Learning Factors (MOLF) and Multiple Gain Adaptation (MGA).

These categories provide distinct strategies for leveraging second-order information in neural network training, enhancing convergence rates while managing computational demands. Figure 1 shows the taxonomy of different second order learning algorithms.

D. Quasi-Newton Methods

Quasi-Newton methods [1], [4] are designed to approximate the Hessian matrix or its inverse using gradient information, enhancing convergence speeds without the full computational burden of Newton's method. Key algorithms include:

- **BFGS (Broyden-Fletcher-Goldfarb-Shanno):** Updates an approximation of the inverse Hessian matrix using gradient and position changes.
- **L-BFGS (Limited-memory BFGS):** [20] A memory-efficient variant of BFGS, particularly suited for large-scale optimization problems.
- **Stochastic L-BFGS:** [17] Adapts L-BFGS for stochastic settings, integrating variance reduction techniques for noisy gradient estimates.
- **Stochastic Block BFGS:** [23] Further divides Hessian approximations into manageable sub-blocks to improve scalability.

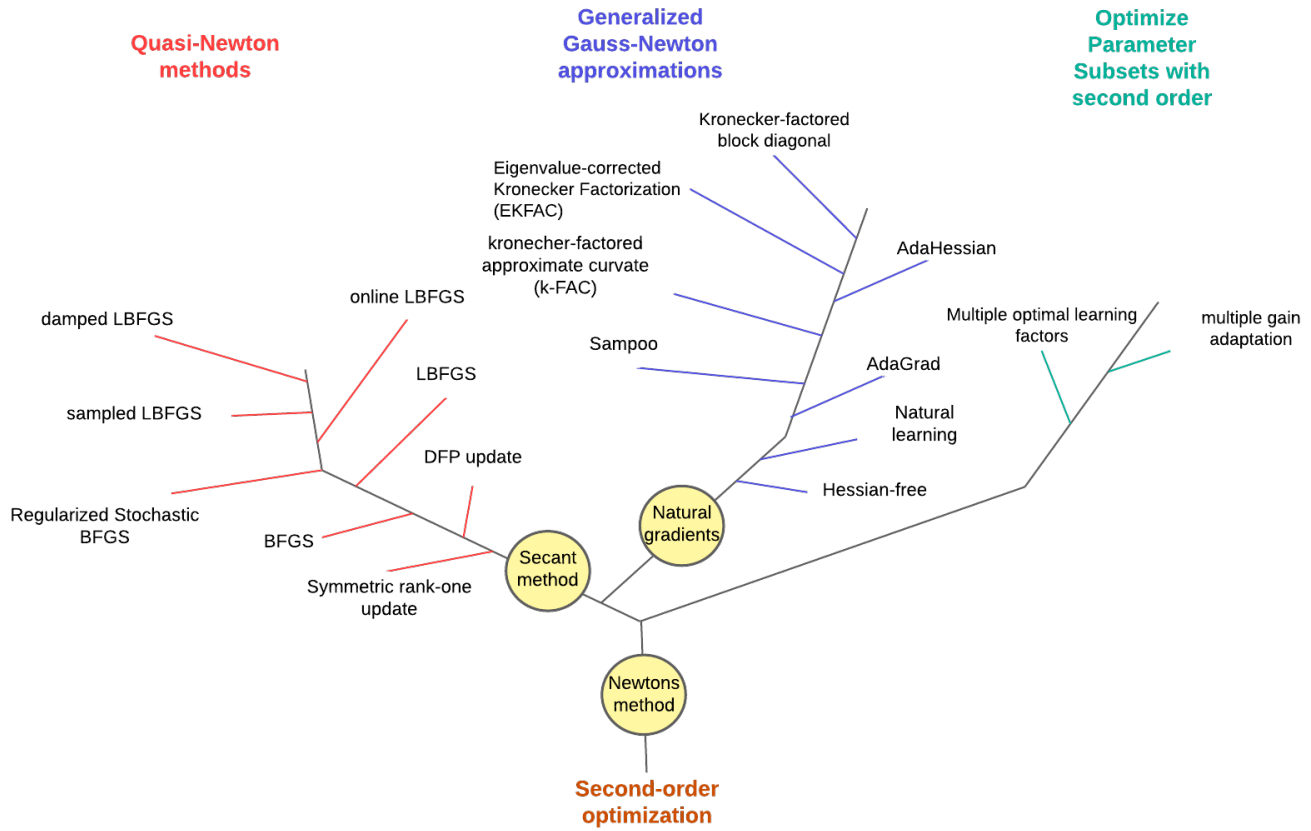


Fig. 1. Taxonomy of Second-Order Optimization Methods for Neural Networks

E. Generalized Gauss-Newton Methods

Generalized Gauss-Newton (GGN) methods [7], [21], [49] approximate the Hessian by focusing on the curvature along residuals of the loss function, effectively capturing the problem's structure without fully computing second derivatives [10], [13], [27]. These methods are particularly effective in deep learning applications with non-convex loss landscapes. Key algorithms include:

- **Shampoo:** Employs block-diagonal Hessian approximations with efficient gradient preconditioning.
- **K-FAC (Kronecker-Factored Approximate Curvature):** [22], [36], [41] Approximates the Fisher Information Matrix using Kronecker-factored blocks for scalability.
- **EKFAC (Empirical Kronecker-Factored Approximate Curvature):** Empirically adjusts curvature approximations to improve accuracy and reduce computational cost.
- **AdaGrad:** [11], [15] Adapts learning rates based on the geometry of observed data, implicitly incorporating second-order information.
- **AdaHessian:** [40] Dynamically adjusts the Hessian approximation with adaptive learning rates, enhancing performance in complex, non-convex settings.

F. Parameter Subset Optimization Methods

Parameter Subset Optimization Methods apply second-order techniques selectively to specific subsets of parameters, such as learning rates, scaling factors, or specific layer weights. This targeted approach allows these methods to capture second-order benefits without the full computational cost. Prominent algorithms include:

- **Multiple Gain Adaptation (MGA):** [37] [39] Optimizes gain factors for clustered weights using Newton's method, dynamically adjusting clusters based on curvature.
- **Multiple Optimal Learning Factors (MOLF):** [34] Computes individualized learning factors for each neuron using second-order information, updating all connected weights with these factors.

III. QUASI-NEWTON METHODS

Quasi-Newton methods are iterative optimization techniques designed to find the local minima or maxima of functions, especially in large-scale problems where computing the full Hessian matrix, as required in Newton's method, is computationally prohibitive. These methods approximate the Hessian matrix (or its inverse) using gradient evaluations, thereby incorporating second-order information about the curvature of the function without the full cost of computing second derivatives [9], [24], [28], [29].

The core idea of Quasi-Newton methods is to iteratively build an approximation of the inverse Hessian matrix using gradient information [50]–[52]. Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be the objective function to minimize. The general iterative update rule for Quasi-Newton methods is given by:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \hat{\mathbf{H}}_k^{-1} \nabla f(\mathbf{x}_k), \quad (3)$$

where:

- $\mathbf{x}_k$ is the current point in the parameter space.
- α_k is the step size (learning rate), which can be determined using line search methods.
- $\nabla f(\mathbf{x}_k)$ is the gradient of the function at $\mathbf{x}_k$.
- $\hat{\mathbf{H}}_k^{-1}$ is the approximation of the inverse Hessian matrix at iteration k .

Updating the Inverse Hessian Approximation

In Quasi-Newton methods, the Hessian approximation $\hat{\mathbf{H}}_k$ is updated iteratively using information from the gradient and position differences between successive iterations [8]. This approach avoids the direct computation of second derivatives. The update of the inverse Hessian approximation relies on two main quantities:

- **Gradient Difference:**

$$\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k), \quad (4)$$

which represents the change in gradients between the current and the previous steps.

- **Position Difference:**

$$\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k, \quad (5)$$

which represents the change in position between the current and the previous steps.

The update rule for the inverse Hessian approximation, $\hat{\mathbf{H}}_{k+1}^{-1}$, generally has the form:

$$\hat{\mathbf{H}}_{k+1}^{-1} = \hat{\mathbf{H}}_k^{-1} + \text{Correction Terms}, \quad (6)$$

where the correction terms are derived to ensure that the new approximation satisfies the secant equation:

$$\hat{\mathbf{H}}_{k+1}^{-1} \mathbf{y}_k = \mathbf{s}_k. \quad (7)$$

This secant equation is fundamental to Quasi-Newton methods, as it incorporates the curvature information captured by the differences in gradients and positions, ensuring that the updated approximation aligns with the observed changes and effectively captures the local curvature of the objective function.

A. Advantages of Quasi-Newton Methods

Quasi-Newton methods offer several advantages over other optimization techniques:

- **Superlinear Convergence:** Quasi-Newton methods typically achieve faster convergence rates compared to first-order methods like gradient descent, thanks to their use of curvature information.

- **Reduced Computation Cost:** By approximating the Hessian matrix instead of computing it directly, Quasi-Newton methods significantly reduce the computational burden associated with second-order methods like Newton's method.
- **Memory Efficiency:** Unlike methods that require the storage of the full Hessian, Quasi-Newton methods, especially in their limited-memory forms, store only a small number of vectors, thereby reducing memory requirements.
- **Adaptive Step Sizes:** The updates inherently adapt the step sizes according to the curvature of the objective function, making Quasi-Newton methods robust and efficient in a wide range of optimization scenarios.

Below, we detail some of the prominent Quasi-Newton algorithms used in neural network training.

B. Prominent Quasi-Newton Algorithms

1) *BFGS (Broyden-Fletcher-Goldfarb-Shanno)*: The BFGS algorithm is one of the most widely used Quasi-Newton methods for optimization. It iteratively updates an approximation of the inverse Hessian matrix using gradient and position changes to guide the search direction. The update rule for the inverse Hessian approximation $\hat{\mathbf{H}}_{k+1}^{-1}$ is given by:

$$\hat{\mathbf{H}}_{k+1}^{-1} = \left(\mathbf{I} - \frac{\mathbf{s}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \right) \hat{\mathbf{H}}_k^{-1} \left(\mathbf{I} - \frac{\mathbf{y}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} \right) + \frac{\mathbf{s}_k \mathbf{s}_k^T}{\mathbf{y}_k^T \mathbf{s}_k}, \quad (8)$$

where:

- $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$ is the position difference.
- $\mathbf{y}_k = \nabla f(\mathbf{x}_{k+1}) - \nabla f(\mathbf{x}_k)$ is the gradient difference.

BFGS ensures the positive definiteness of the Hessian approximation and adapts effectively to the local curvature of the loss function, which enhances convergence speed compared to first-order methods.

2) *L-BFGS (Limited-memory BFGS)*: L-BFGS [3], [20] is a memory-efficient variant of BFGS designed for high-dimensional problems frequently encountered in deep learning and large-scale optimization tasks. Instead of storing the full Hessian approximation, L-BFGS retains only a limited number of recent updates, significantly reducing memory usage. The method approximates the inverse Hessian using pairs of vectors $\{\mathbf{s}_i, \mathbf{y}_i\}$ from the most recent iterations.

The update process employs a two-loop recursion, which efficiently computes the product of the inverse Hessian approximation with a vector without explicitly forming the full matrix:

$$\hat{\mathbf{H}}_k^{-1} \approx \text{Two-loop recursion using stored pairs}, \quad (9)$$

The two-loop recursion can be summarized as follows:

- **First Loop:** Computes an intermediate vector by scaling and adjusting the previous approximations using stored pairs $\{\mathbf{s}_i, \mathbf{y}_i\}$. Scaling factors $\rho_i = 1/(\mathbf{y}_i^T \mathbf{s}_i)$ are calculated and iteratively applied.

- **Second Loop:** Refines the intermediate vector by applying adjustments in reverse order, ultimately computing the inverse Hessian-vector product needed for the optimization step.

L-BFGS effectively incorporates second-order information with minimal memory and computational overhead, making it particularly suitable for large-scale optimization tasks.

3) *Stochastic L-BFGS*: Stochastic L-BFGS [25] is a stochastic variant of L-BFGS tailored for noisy gradient environments common in stochastic optimization. The algorithm updates the inverse Hessian approximation using mini-batches of data, incorporating stochastic corrections that reduce variance [19]. This enhances performance over standard stochastic gradient descent (SGD). The update rule incorporates these stochastic corrections:

$$\hat{\mathbf{H}}_{k+1}^{-1} = \hat{\mathbf{H}}_k^{-1} + \text{Stochastic corrections based on mini-batches.} \quad (10)$$

Stochastic L-BFGS is particularly effective for large-scale, non-convex optimization problems such as training deep neural networks, where traditional methods struggle with noise in gradient estimates.

4) *Stochastic Block BFGS*: Stochastic Block BFGS is a stochastic variant of the BFGS algorithm designed for large-scale optimization problems. This method extends traditional BFGS by using stochastic sketches of the Hessian, allowing it to incorporate richer curvature information with reduced computational overhead. The algorithm employs subsampled Hessian information and randomized sketching techniques to maintain a limited-memory approximation of the inverse Hessian.

The update rule for the inverse Hessian approximation $\hat{\mathbf{H}}_{k+1}^{-1}$ in Stochastic Block BFGS is defined as:

$$\hat{\mathbf{H}}_{k+1}^{-1} = \mathbf{D}_k \Delta_k \mathbf{D}_k^T + (\mathbf{I} - \mathbf{D}_k \Delta_k \mathbf{Y}_k^T) \hat{\mathbf{H}}_k^{-1} (\mathbf{I} - \mathbf{Y}_k \Delta_k \mathbf{D}_k^T), \quad (11)$$

where:

- $\mathbf{D}_k$ is a randomly generated matrix with columns representing sketching directions or past search directions.
- $\mathbf{Y}_k = \nabla^2 f(\mathbf{x}_k) \mathbf{D}_k$ represents the sketched subsampled Hessian matrix.
- $\Delta_k = (\mathbf{D}_k^T \mathbf{Y}_k)^{-1}$ is a scaling factor computed from the inner products of $\mathbf{D}_k$ and $\mathbf{Y}_k$.

Stochastic Block BFGS utilizes a two-loop recursion similar to L-BFGS, efficiently computing the inverse Hessian-vector product without explicitly storing the large matrices. This approach captures more curvature information compared to single-vector updates, enhancing convergence rates, particularly in high-dimensional settings typical of machine learning applications.

- **Enhanced Curvature Information:** By using multiple directions in each update, Stochastic Block BFGS captures a broader spectrum of curvature information compared to traditional methods.

- **Memory Efficiency:** The algorithm's limited-memory approach allows it to handle large-scale problems without the need for full Hessian storage.
- **Robust Convergence:** The combination of stochastic updates and variance reduction techniques, such as SVRG, leads to faster and more stable convergence.

This algorithm is particularly effective for empirical risk minimization problems in deep learning, where high dimensionality and large datasets make standard quasi-Newton methods impractical.

IV. GENERALIZED GAUSS-NEWTON METHOD AND NATURAL GRADIENTS

The Generalized Gauss-Newton (GGN) method is an advanced second-order optimization technique widely used in training neural networks, particularly in scenarios where traditional first-order methods like stochastic gradient descent struggle with complex loss landscapes. GGN methods approximate the Hessian matrix in a computationally efficient manner, leveraging the structure of the problem to capture essential curvature information. These methods are closely related to Natural Gradient [6], [32] methods, which adjust the gradient by accounting for the geometry of the parameter space. Below, we provide a detailed explanation of these methods with relevant equations.

A. Generalized Gauss-Newton Methods

The Generalized Gauss-Newton (GGN) method approximates the Hessian matrix by focusing on the loss function's curvature along the residuals rather than directly calculating second derivatives, which can be computationally prohibitive. This approximation is particularly useful for non-linear least squares problems, such as those encountered in deep learning.

Let $f(\mathbf{x})$ be the objective function, and suppose that it can be decomposed into a sum of residuals:

$$f(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^N r_i(\mathbf{x})^2, \quad (12)$$

where $r_i(\mathbf{x})$ are the residuals. The gradient of $f(\mathbf{x})$ is given by:

$$\nabla f(\mathbf{x}) = \sum_{i=1}^N r_i(\mathbf{x}) \nabla r_i(\mathbf{x}). \quad (13)$$

The Hessian matrix $\mathbf{H}$ can be decomposed as:

$$\mathbf{H} = \sum_{i=1}^N [\nabla r_i(\mathbf{x}) \nabla r_i(\mathbf{x})^T + r_i(\mathbf{x}) \nabla^2 r_i(\mathbf{x})]. \quad (14)$$

In GGN, the Hessian approximation $\mathbf{H}_{\text{GGN}}$ ignores the second term, which involves second derivatives of the residuals, focusing only on the outer product of the Jacobians:

$$\mathbf{H}_{\text{GGN}} = \sum_{i=1}^N \nabla r_i(\mathbf{x}) \nabla r_i(\mathbf{x})^T = \mathbf{J}^T \mathbf{J}, \quad (15)$$

Below, we explain some of the prominent Generalized Gauss-Newton algorithms used in neural network training.

B. Prominent GGN algorithms

1) *Shampoo*: Shampoo [31] is a preconditioned stochastic optimization algorithm designed for tensor spaces, where each parameter block in the neural network is treated separately. Instead of forming a full preconditioning matrix, Shampoo maintains a set of smaller preconditioning matrices for each dimension, making it computationally efficient. The update rule for Shampoo in the matrix case is:

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \eta \mathbf{L}_t^{-1/4} \mathbf{G}_t \mathbf{R}_t^{-1/4}, \quad (16)$$

where:

- $\mathbf{L}_t = \mathbf{L}_{t-1} + \mathbf{G}_t \mathbf{G}_t^T$ is the left preconditioner matrix.
- $\mathbf{R}_t = \mathbf{R}_{t-1} + \mathbf{G}_t^T \mathbf{G}_t$ is the right preconditioner matrix.
- $\mathbf{G}_t$ is the gradient at time t .

Shampoo's approach efficiently captures second-order information while maintaining low computational overhead, making it suitable for large-scale neural network training.

2) *K-FAC (Kronecker-Factored Approximate Curvature)*: K-FAC [18], [26], [33] is a layer-wise approximation method that approximates the Fisher Information Matrix (FIM) by assuming independence between the gradients of different layers. This results in a Kronecker-factored structure that reduces the computational burden associated with calculating the full Hessian. The update rule involves using a Kronecker product of smaller matrices:

$$\mathbf{F} \approx \mathbf{A} \otimes \mathbf{B}, \quad (17)$$

where $\mathbf{A}$ and $\mathbf{B}$ are the approximations of the layer-wise statistics of the input and gradient, respectively. This method significantly reduces the complexity from $O(n^3)$ to manageable computations, making it practical for deep learning applications.

3) *EKFAC (Empirical Kronecker-Factored Approximate Curvature)*: EKFAC [14], [16], [30] is a variant of K-FAC that empirically adjusts the Kronecker factors based on the observed curvature, making it more adaptive to non-stationary training conditions. EKFAC retains the Kronecker-factored structure but introduces updates based on empirical measurements of gradient variance:

$$\mathbf{H}_{\text{EKFAC}} = \mathbf{A}_{\text{empirical}} \otimes \mathbf{B}_{\text{empirical}}, \quad (18)$$

where $\mathbf{A}_{\text{empirical}}$ and $\mathbf{B}_{\text{empirical}}$ are updated using real-time data rather than fixed assumptions, improving robustness and convergence stability.

4) *AdaHessian*: AdaHessian [40] is a second-order stochastic optimization algorithm designed to dynamically incorporate the curvature of the loss function using adaptive estimates of the Hessian. Unlike traditional second-order methods that are computationally expensive, AdaHessian employs a series of novel techniques to maintain low computational overhead while leveraging curvature information effectively. Key innovations include Hutchinson's method for efficient Hessian diagonal

approximation, exponential moving averages to smooth Hessian estimates, and block diagonal averaging to reduce variance in curvature estimates.

The update rule for AdaHessian can be expressed as:

$$\hat{\mathbf{H}}_t = \sqrt{(1 - \beta) \sum_{i=1}^t \beta^{t-i} D_i D_i^T}, \quad (19)$$

where:

- β is the decay rate controlling the influence of past Hessian information.
- D_i is the spatially averaged Hessian diagonal at step i , approximated using Hutchinson's method.

AdaHessian updates the parameter vector $\mathbf{x}_t$ using the adaptive preconditioning:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \hat{\mathbf{H}}_t^{-1} \nabla f(\mathbf{x}_t), \quad (20)$$

where η is the learning rate. By efficiently approximating the Hessian diagonal, AdaHessian scales gradient updates according to the curvature of the loss landscape, improving convergence rates and robustness across a variety of machine learning tasks.

5) *AdaGrad*: AdaGrad [11] is an adaptive gradient algorithm that scales learning rates based on the geometry of the observed data. It adjusts learning rates individually for each parameter based on the accumulation of past gradient magnitudes, effectively handling sparse and non-stationary data. The update rule for AdaGrad is given by:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{\eta}{\sqrt{\mathbf{G}_t + \epsilon}} \nabla f(\mathbf{x}_t), \quad (21)$$

where $\mathbf{G}_t = \sum_{i=1}^t \nabla f(\mathbf{x}_i) \nabla f(\mathbf{x}_i)^T$ accumulates the squared gradients over time, and ϵ is a small constant to prevent division by zero. AdaGrad's ability to adaptively adjust learning rates makes it particularly effective for problems where certain features are informative but rarely seen.

These methods efficiently balance computational complexity and second-order information, making them popular in deep learning applications where optimization efficiency is critical.

V. PARAMETER SUBSET OPTIMIZATION METHODS

Parameter Subset Optimization Methods apply second-order optimization techniques selectively to specific subsets of neural network parameters, such as learning rates, scaling factors, or weights of particular layers. This targeted approach allows these methods to capture the benefits of second-order information without the significant computational cost of applying it to the entire parameter set. By focusing on key subsets of parameters, these methods provide a balance between optimization efficiency and performance [38]. Below, we describe two prominent algorithms in this category: Multiple Optimal Learning Factors (MOLF) and Multiple Gain Adaptation (MGA).

A. Prominent Algorithms

1) *Multiple Optimal Learning Factors (MOLF)*: Multiple Optimal Learning Factors (MOLF) is a second-order optimization algorithm that calculates an individual learning factor for each neuron in a neural network. This method uses second-order information to determine these learning factors, which are then used to update all the weights connected to the corresponding neuron. This neuron-specific adjustment helps to optimize the learning process by adapting the learning rates according to the local curvature of the loss function.

a) *Algorithm Overview*: MOLF employs a diagonal transformation of the net function vector of the multilayer perceptron (MLP), assigning a unique learning factor to each neuron. The update equations for MOLF are derived from the gradients and Hessians specific to each neuron's learning factor:

$$\mathbf{H}_{\text{MOLF}} \cdot \mathbf{z} = \mathbf{g}_{\text{MOLF}}, \quad (22)$$

where:

- $\mathbf{H}_{\text{MOLF}}$ is the Hessian matrix associated with the learning factors.
- $\mathbf{g}_{\text{MOLF}}$ is the gradient vector.
- $\mathbf{z}$ is the vector of optimal learning factors.

The input weights connecting each input to the hidden units are updated using the calculated learning factors. For the k -th hidden unit, the update rule is:

$$w_{k,n} \leftarrow w_{k,n} + z_k g_{k,n}, \quad (23)$$

where z_k is the learning factor for the k -th hidden unit, and $g_{k,n}$ is the gradient of the error with respect to the weight $w_{k,n}$. MOLF's use of individualized learning rates allows it to tailor updates, enhancing convergence and efficiency in training complex neural network architectures.

2) *Multiple Gain Adaptation (MGA)*: Multiple Gain Adaptation (MGA) optimizes gain factors for input weights by grouping weights into clusters based on the curvature of the Loss function.

a) *Algorithm Overview*: MGA operates by dynamically adjusting the number of clusters of input weights connected to each hidden unit based on the curvature of the loss function. Weights with similar curvatures are grouped together, and a single gain factor is calculated for each cluster using Newton's method.

The number of clusters is dynamically adjusted throughout training by monitoring the loss change per floating point operation (FLOP). When the loss change per FLOP increases, the number of clusters is increased, allowing more precise adjustments with additional gain factors. Conversely, when the loss change per FLOP decreases, the number of clusters is reduced, optimizing computational efficiency without compromising performance.

B. Benefits and Challenges

MGA and MOLF effectively leverage second-order information to optimize subsets of neural network parameters, providing significant benefits:

- **Enhanced Convergence**: By adapting learning rates according to local curvature, these methods can achieve faster convergence compared to traditional first-order methods.
- **Scalability**: The computational burden is reduced by focusing second-order updates on select parameters rather than the entire network.
- **Flexibility**: The algorithms adjust their focus dynamically, facilitating efficient training across varying network sizes and architectures.

However, challenges include tuning parameters associated with cluster sizes and managing computational complexity, particularly when scaling to large networks. These methods are still evolving to address issues related to stability and efficient implementation in diverse neural network architectures.

VI. RESULTS

To assess the effectiveness of various second-order optimization algorithms, we conducted experiments on the CIFAR-100 dataset using five prominent algorithms: Shampoo, K-FAC, L-BFGS, AdaGrad, and MOLF. The CIFAR-100 dataset, comprising 60,000 images across 100 classes [35], serves as a challenging benchmark for image classification tasks. Each algorithm was evaluated on ResNet-50 architecture, and the results were evaluated based on test accuracy, training steps per second, and the number of epochs required to reach 90% of the peak test accuracy.

A. Experimental Setup

The experiments were conducted using a ResNet-50 [45] model trained on 8 A100 GPUs, with each algorithm trained for 200 epochs using a batch size of 128. Initial learning rates were tuned individually for each algorithm to optimize performance. The following repositories were used for the implementations of the optimization algorithms:

- **K-FAC**: Implemented from "Scalable K-FAC Training for Deep Neural Networks with Distributed Preconditioning" [42], GitHub: https://github.com/lzhangbv/kfac_pytorch.
- **L-BFGS**: Implemented from "Implementation of Stochastic Quasi-Newton's Method in PyTorch" [43], GitHub: <https://github.com/harryliaw/SdLBFGS>.
- **Shampoo**: Implemented from "Scalable Second Order Optimization for Deep Learning" [44], GitHub: https://github.com/google-research/google-research/tree/master/scalable_shampoo_pytorch.
- **MOLF**: Implemented from "Multiple Optimal Learning Factors for the Multi-layer Perceptron" [34].

The primary metrics recorded included test accuracy, training steps per second, and the number of epochs needed to achieve 90% of the maximum test accuracy observed during training. Detailed experimental settings and code are available at <https://github.com/Jasmamu1992/second-order-optimization>.

B. Comparison of Results

Table I summarizes the comparative performance of the five algorithms on the CIFAR-100 dataset, highlighting key metrics such as validation accuracy, training speed, and convergence rate.

TABLE I
PERFORMANCE COMPARISON OF OPTIMIZATION ALGORITHMS ON CIFAR-100

Algorithm	Val Accuracy (%)	Training Steps per Second	90th Percentile Epoch
Shampoo	75.93	1.24	79
K-FAC	76.39	1.02	56
L-BFGS	76.23	0.92	65
AdaGrad	75.63	1.26	93
MOLF	75.83	1.13	88

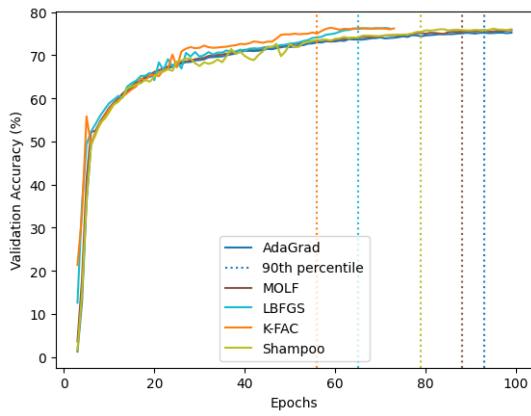


Fig. 2. Validation Accuracy (%) vs. Epochs for Different Optimization Algorithms

Figure 2 illustrates the validation accuracy curves for five algorithms on the CIFAR-100 dataset, highlighting the epoch at which each algorithm achieves 90th percentile validation accuracy.

C. Performance Analysis

The results demonstrate that K-FAC achieved the highest validation accuracy and fastest convergence, reaching 90% of its peak accuracy by epoch 56. This performance can be attributed to K-FAC's structured approximation of the Fisher Information Matrix, which allows for efficient second-order updates that enhance convergence speed without excessive computational overhead. Shampoo also performed well, offering robust accuracy and training speed due to its block-diagonal approximations and efficient gradient preconditioning strategies, balancing computational cost and convergence rates effectively.

L-BFGS showed competitive accuracy, but its limited-memory approach required more epochs to converge compared to K-FAC and Shampoo. This highlights a trade-off between memory efficiency and convergence speed, as L-BFGS maintains only a subset of curvature information, which can delay reaching optimal performance. MOLF, which computes individualized learning factors for each neuron, delivered solid

performance but slower convergence, reflecting the challenges of dynamically adapting learning rates in complex networks.

AdaGrad, despite being the fastest in terms of training steps per second, lagged behind in both convergence speed and final accuracy. Its adaptive learning rate strategy, while effective for sparse data, does not fully leverage second-order information, which can limit its efficacy in navigating the intricate landscapes of deep neural network loss functions.

These results highlight the strengths of structured second-order methods like K-FAC and Shampoo in efficiently optimizing deep neural networks. The balance between computational efficiency and the depth of curvature information captured by these methods allows them to excel in complex, high-dimensional settings, underscoring their potential to improve training dynamics significantly.

VII. CHALLENGES AND FUTURE DIRECTIONS

Second-order optimization methods have significantly advanced the training of deep neural networks by leveraging curvature information, offering faster convergence and improved performance compared to traditional first-order methods. Despite these advances, several challenges remain, particularly concerning computational efficiency, memory demands, and the dynamic nature of neural network training.

Many algorithms discussed in this paper have made considerable strides in overcoming key obstacles. For instance, methods like Shampoo, K-FAC, and AdaHessian have introduced novel techniques to approximate the Hessian or Fisher Information Matrix, reducing the prohibitive computational costs traditionally associated with second-order methods. Shampoo, by leveraging block-diagonal approximations, manages to capture rich curvature information with a manageable computational load, making it feasible for large-scale models. K-FAC employs a layer-wise Kronecker-factored approximation that significantly reduces the computational burden, allowing second-order updates to scale efficiently with deep architectures. Similarly, AdaHessian adapts the Hessian dynamically, focusing on the curvature's most informative directions, which helps balance computational demands with the need for robust second-order information.

Despite these advances, challenges persist. A critical issue is the scalability of these methods when applied to extremely large networks or high-dimensional datasets. While block-diagonal and Kronecker-factored approximations reduce computational complexity, they can still become bottlenecks when the number of parameters is exceptionally high, such as in transformer models or other state-of-the-art architectures. Furthermore, maintaining these approximations requires significant memory, which can strain resources in environments with limited capacity, such as edge devices or when training very large models on standard hardware.

Another ongoing challenge is the sensitivity of second-order methods to hyperparameter settings. Algorithms like K-FAC and Shampoo rely on careful tuning of learning rates, damping factors, and preconditioning parameters. These hyperparameters are crucial for ensuring stability and convergence, yet they often

require extensive manual tuning or heuristic adjustments. This sensitivity can lead to suboptimal performance if not carefully managed, particularly in the early stages of training where the curvature information may be noisy or misleading.

Moreover, the dynamic and non-stationary nature of neural network loss landscapes complicates the effective use of curvature information. Many second-order methods assume relatively stable curvature, but in practice, the training dynamics of deep networks can cause rapid changes in the curvature, leading to outdated or inaccurate approximations. Algorithms like AdaHessian attempt to address this by dynamically updating their Hessian estimates, but these updates can still lag behind the actual changes in the loss landscape, limiting their effectiveness in highly non-convex settings.

Future research should focus on developing more adaptive and scalable second-order methods that can efficiently update curvature approximations in real-time, keeping pace with the evolving loss landscape of neural networks. Hybrid approaches that combine first-order and second-order information selectively, applying full second-order updates only where they are most beneficial, could further enhance efficiency. Additionally, advances in automated hyperparameter tuning, such as Bayesian optimization or reinforcement learning-based tuning, could alleviate the sensitivity issues that currently hamper second-order methods.

To fully harness the potential of second-order optimization in deep learning, continued innovation is needed in reducing computational overhead, improving adaptability to non-stationary environments, and simplifying integration into existing deep learning frameworks. By addressing these remaining challenges, second-order methods can become more accessible and widely applicable, driving further improvements in neural network training and performance.

REFERENCES

- [1] Shanno, D. Conditioning of quasi-Newton methods for function minimization. *Mathematics Of Computation*. **24**, 647-656 (1970)
- [2] Becker, S., Le Cun, Y. & Others Improving the convergence of back-propagation learning with second order methods. *Proceedings Of The 1988 Connectionist Models Summer School*. pp. 29-37 (1988)
- [3] Liu, D. & Nocedal, J. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*. **45**, 503-528 (1989)
- [4] Byrd, R., Nocedal, J. & Schnabel, R. Representations of quasi-Newton matrices and their use in limited memory methods. *Mathematical Programming*. **63**, 129-156 (1994)
- [5] LeCun, Y., Bottou, L., Orr, G. & Muller, K. Efficient backprop. *Neural Networks: Tricks Of The Trade*. New York: Springer. (1998)
- [6] Amari, S., Park, H. & Fukumizu, K. Adaptive method of realizing natural gradient learning for multilayer perceptrons. *Neural Computation*. **12**, 1399-1409 (2000)
- [7] Heskes, T. On "Natural" Learning and Pruning in Multilayered Perceptrons. *Neural Computation*. **12** (2000,1)
- [8] Schraudolph, N., Yu, J. & Günter, S. A stochastic quasi-Newton method for online convex optimization. *Artificial Intelligence And Statistics*. pp. 436-443 (2007)
- [9] Bordes, A., Bottou, L. & Gallinari, P. SGD-QN: Careful quasi-Newton stochastic gradient descent. *Journal Of Machine Learning Research*. **10**, 1737-1754 (2009)
- [10] Martens, J. Deep learning via hessian-free optimization.. *ICML*. **27** pp. 735-742 (2010)
- [11] Duchi, J., Hazan, E. & Singer, Y. Adaptive subgradient methods for online learning and stochastic optimization. *Journal Of Machine Learning Research*. **12**, 2121-2159 (2011)
- [12] Hinton, G., Srivastava, N. & Swersky, K. Neural networks for machine learning lecture 6a overview of mini-batch gradient descent. *Cited On*. **14** (2012)
- [13] Kiros, R. Training neural networks with stochastic Hessian-free optimization. *ArXiv Preprint ArXiv:1301.3641*. (2013)
- [14] Johnson, R. & Zhang, T. Accelerating stochastic gradient descent using predictive variance reduction. *Advances In Neural Information Processing Systems*. pp. 315-323 (2013)
- [15] Kingma, D. & Ba, J. Adam: A Method for Stochastic Optimization. *International Conference On Learning Representations*. (2014)
- [16] Martens, J. New insights and perspectives on the natural gradient method. *ArXiv Preprint ArXiv:1412.1193*. (2014)
- [17] Mokhtari, A. & Ribeiro, A. RES: Regularized stochastic BFGS algorithm. *IEEE Transactions On Signal Processing*. **62**, 6089-6104 (2014)
- [18] Martens, J. & Grosse, R. Optimizing neural networks with kronecker-factored approximate curvature. *International Conference On Machine Learning*. pp. 2408-2417 (2015)
- [19] Lucchi, A., McWilliams, B. & Hofmann, T. A variance reduced stochastic newton method. *ArXiv Preprint ArXiv:1503.08316*. (2015)
- [20] Mokhtari, A. & Ribeiro, A. Global convergence of online limited memory BFGS. *The Journal Of Machine Learning Research*. **16**, 3151-3181 (2015)
- [21] Desjardins, G., Simonyan, K., Pascanu, R. & Others Natural neural networks. *Advances In Neural Information Processing Systems*. pp. 2071-2079 (2015)
- [22] Grosse, R. & Martens, J. A kronecker-factored approximate fisher matrix for convolution layers. *International Conference On Machine Learning*. pp. 573-582 (2016)
- [23] Gower, R., Goldfarb, D. & Richtárik, P. Stochastic block BFGS: Squeezing more curvature out of data. *International Conference On Machine Learning*. pp. 1869-1878 (2016)
- [24] Byrd, R., Hansen, S., Nocedal, J. & Singer, Y. A stochastic quasi-Newton method for large-scale optimization. *SIAM Journal On Optimization*. **26**, 1008-1031 (2016)
- [25] Moritz, P., Nishihara, R. & Jordan, M. A linearly-convergent stochastic L-BFGS algorithm. *Artificial Intelligence And Statistics*. pp. 249-258 (2016)
- [26] Ba, J., Grosse, R. & Martens, J. Distributed Second-Order Optimization using Kronecker-Factored Approximations. *ICLR*. (2017)
- [27] Botev, A., Ritter, H. & Barber, D. Practical gauss-newton optimisation for deep learning. *Proceedings Of The 34th International Conference On Machine Learning-Volume 70*. pp. 557-565 (2017)
- [28] Wang, X., Ma, S., Goldfarb, D. & Liu, W. Stochastic quasi-Newton methods for nonconvex stochastic optimization. *SIAM Journal On Optimization*. **27**, 927-956 (2017)
- [29] Gower, R. & Richtárik, P. Randomized quasi-Newton updates are linearly convergent matrix inversion algorithms. *SIAM Journal On Matrix Analysis And Applications*. **38**, 1380-1409 (2017)
- [30] George, T., Laurent, C., Bouthillier, X., Ballas, N. & Vincent, P. Fast approximate natural gradient descent in a kronecker factored eigenbasis. *Advances In Neural Information Processing Systems*. pp. 9550-9560 (2018)
- [31] Gupta, V., Koren, T. & Singer, Y. Shampoo: Preconditioned Stochastic Tensor Optimization. *Proceedings Of The 35th International Conference On Machine Learning*. **80** pp. 1842-1850 (2018)
- [32] Li, W. & Montúfar, G. Natural gradient via optimal transport. *Information Geometry*. **1**, 181-214 (2018)
- [33] Kunstner, F., Balles, L. & Hennig, P. Limitations of the Empirical Fisher Approximation. *ArXiv Preprint ArXiv:1905.12558*. (2019)
- [34] Malalur, S., Manry, M., Jesudhas, P. Multiple optimal learning factors for the multi-layer perceptron. *Neurocomputing*. **149** pp. 1490-1501 (2015).
- [35] Ganpathy, G., Shrivastava, N., Vyas, O., Singh, M., Arora, R. & Mishra, S. Intrusion Detection and Attack Classification using an Ensemble Approach. *International Research Journal Of Engineering And Technology*. **7** pp. 1616-1620 (2020)
- [36] Eschenhagen, R., Immer, A., Turner, R., Schneider, F. Hennig, P. Kronecker-Factored Approximate Curvature for Modern Neural Network Architectures. *Advances In Neural Information Processing Systems*. **36** pp. 33624-33655 (2023),
- [37] Challagundla, J., Tyagi, K., Chugh, T. & Manry, M. Multiple Gain Adaptations for Improved Neural Networks Training. *2024 IEEE 14th Annual Computing And Communication Workshop And Conference (CCWC)*. pp. 0414-0420 (2024)

- [38] Chugh, T., Tyagi, K., Srinivasan, P. & Challagundla, J. State-Based Dynamic Graph with Breadth First Progression For Autonomous Robots. *2024 IEEE 14th Annual Computing And Communication Workshop And Conference (CCWC)*. pp. 0365-0369 (2024)
- [39] Challagundla, J. Adaptive Multiple Optimal Learning Factors For Neural Network Training. *M.S. Thesis/Dissertation* <https://rc.library.uta.edu/uta-ir/handle/10106/25030>. 1, 30-49 (2015)
- [40] Yao, Z., Gholami, A., Shen, S., Mustafa, M., Keutzer, K. & Mahoney, M. ADAHESSIAN: An Adaptive Second Order Optimizer for Machine Learning. (2021), <https://arxiv.org/abs/2006.00719>
- [41] Dangel, F., Müller, J. & Zeinhofer, M. Kronecker-Factored Approximate Curvature for Physics-Informed Neural Networks. (2024), <https://arxiv.org/abs/2405.15603>
- [42] Zhang, L., Shi, S., Wang, W. & Li, B. Scalable K-FAC Training for Deep Neural Networks with Distributed Preconditioning. (2022), <https://arxiv.org/abs/2206.15143>
- [43] Li, Y. & Liu, H. Implementation of Stochastic Quasi-Newton's Method in PyTorch. (2018), <https://arxiv.org/abs/1805.02338>
- [44] Anil, R., Gupta, V., Koren, T., Regan, K. & Singer, Y. Scalable Second Order Optimization for Deep Learning. (2021), <https://arxiv.org/abs/2002.09018>
- [45] He, K., Zhang, X., Ren, S. & Sun, J. Deep Residual Learning for Image Recognition. (2015), <https://arxiv.org/abs/1512.03385>
- [46] Ruder, S. An overview of gradient descent optimization algorithms. (2017), <https://arxiv.org/abs/1609.04747>
- [47] Kingma, D. & Ba, J. Adam: A Method for Stochastic Optimization. (2017), <https://arxiv.org/abs/1412.6980>
- [48] Botev, A., Lever, G. & Barber, D. Nesterov's Accelerated Gradient and Momentum as approximations to Regularised Update Descent. (2016), <https://arxiv.org/abs/1607.01981>
- [49] Goldfarb, D., Ren, Y. & Bahamou, A. Practical Quasi-Newton Methods for Training Deep Neural Networks. (2021), <https://arxiv.org/abs/2006.08877>
- [50] Jiang, R., Jin, Q. & Mokhtari, A. Online Learning Guided Curvature Approximation: A Quasi-Newton Method with Global Non-Asymptotic Superlinear Convergence. (2023), <https://arxiv.org/abs/2305.15143>
- [51] Miyaishi, T., Nozawa, R., Poirion, P. & Takeda, A. Sub-space Quasi-Newton Method with Gradient Approximation. (2024), <https://arxiv.org/abs/2406.01965>
- [52] Lin, W., Dangel, F., Eschenhagen, R., Neklyudov, K., Kristiadi, A., Turner, R. & Makhzani, A. Structured Inverse-Free Natural Gradient: Memory-Efficient & Numerically-Stable KFAC. (2024), <https://arxiv.org/abs/2312.05705>